You don’t really need the debugger to work, right? When you develop everything by writing a test first (see chapter 11), and your code doesn’t work, you can change a test or write a new test to see what’s happening. There’s some degree of truth in that, but it’s not practical. Even if you tend not to use it much, one day you’ll need the debugger and you don’t want to be presented with the compiled JavaScript version of your program.

Imagine a browser-based program you wrote—a program, any program, it doesn’t matter which one. Imagine now that you wrote this program in a file called (ingeniously) program.coffee. What do you see if the program throws an exception? You see text something like so:

```
Reference Error: x is not defined        -- program.js 23
```

This is telling you that your program has thrown a reference error in line 23 of program.js. That’s nice, except that you didn’t write any program.js because you wrote your program in CoffeeScript! So what’s the problem?

The problem is not that CoffeeScript is compiled. There are many compiled languages with good debuggers. The problem is that the JavaScript runtime has no knowledge of your original CoffeeScript source, so it has no way to show you where the problem is in your original source. That changes with source maps. Source maps allow the CoffeeScript compiler to tell the JavaScript runtime how source lines in the JavaScript correspond to source lines in the original CoffeeScript source.

13.4.2. Getting started with source maps

To try CoffeeScript with source maps, you’ll need both a runtime that supports them and a CoffeeScript compiler that can create a source map. Right now, that means a CoffeeScript compiler version greater than 1.6 and a recent version of your web browser that supports source maps.[1] Before long, source maps will be supported by more runtimes.

[1] Consult the documentation for your browser to determine if it supports source maps and how you can enable them.